

Author

- **Name:** Giridharen G
- **Roll Number:** 23f2004871
- **Student Email:** 23f2004871@ds.study.iitm.ac.in
- **About:** A Tech enthusiast with hands-on experience in machine learning, quantum-safe cryptography, autonomous systems, and full-stack development.

Description

I understood the project as a full-stack, multi-user web application for managing vehicle parking. The system needed to allow an admin user to manage lots and a regular user to book spots. I built it using the Flask framework with a programmatically generated SQLite database.

- **AI/LLM Used:** Approximately 10-15%. I used an AI assistant specifically to help structure and refactor the API endpoints using the Flask-RESTful extension.

Technologies Used

- **Flask & Extensions:**
 - **Flask:** The main backend framework for handling routing, requests, and application logic.
 - **Flask-SQLAlchemy:** Used as an ORM to model the database schema in Python and interact with the SQLite database.
 - **Flask-Login:** Provided a secure system for managing user sessions and role-based access control.
 - **Flask-RESTful:** Used to build a clear, resource-based JSON API.
 - **Flask-Mail:** Implemented to send asynchronous email notifications for registration and bookings.
- **Database:**
 - **SQLite:** The necessary lightweight, file-based database for data storage.
- **Frontend:**
 - **HTML5 & Jinja2:** For structuring and rendering dynamic web pages on the server side.
 - **CSS3 & Bootstrap 5:** To create a modern, fully responsive, and attractive user interface.
 - **JavaScript (Vanilla & Libraries):** Used for client-side interactivity, including the Leaflet.js library for the interactive map and Chart.js for data visualization.
- **Other Python Libraries:**
 - **qrcode[pil]:** For generating unique QR codes for each reservation.

DB Schema Design

The database had a relational, normalized structure to ensure data integrity and scalability.

- **Structure:** The schema includes seven interconnected models: User, ParkingPlace, ParkingSpot, Reservation, Vehicle, Payment, and Review. Each model is defined as a Python class using Flask-SQLAlchemy.
- **Columns & Constraints:** Each column is strongly typed (e.g., db.Integer, db.String, db.DateTime) with appropriate rules like primary_key=True, unique=True, and nullable=False to enforce data standards. Foreign keys (db.ForeignKey) connect tables, and cascade options (cascade="all, delete-orphan") make sure that related data, like spots, is removed cleanly when a parent entity,

like a lot, is deleted.

- **Design Rationale:** This design was chosen to separate tasks and avoid data repetition. For example, storing vehicle information in a separate Vehicle table and linking it to a reservation through a foreign key is more efficient than including vehicle details in every reservation record. This approach makes the application easier to manage and extend.

API Design

A RESTful API was created to provide programmatic access to application data, mainly for the application's frontend features.

- **Implementation:** The API was built using the **Flask-RESTful** extension, which offers a clear, class-based approach. Each API endpoint corresponds to a Resource class (e.g., LotListResource, LotResource). Data serialization is handled by `marshal_with` decorators and fields dictionaries to ensure consistent JSON output format.
- **Endpoints Created:**
 - **GET /api/lots:** Returns a list of all parking lots.
 - **GET /api/lots/<lot_id>:** Returns detailed information for a single lot, including a list of its spots.
 - **GET /api/lots/<lot_id>/availability:** Provides a live count of available spots by type (Standard, Premium, EV), used for the real-time user dashboard.

Architecture and Features

- **Architecture:** The project uses a modular and scalable design. The main `app.py` file manages initialization and configuration. All application logic is organized into **controllers** (Flask Blueprints) for authentication, user routes, admin routes, and the API. Database models are stored in a separate models directory, and all frontend files are organized in the templates and static directories. This separation keeps the codebase easy to navigate and maintain.
- **Features Implemented:**
 - **Core Features:** All required features were met, including a dual-role (Admin/User) system, full CRUD management for parking lots, and an automated spot allocation system for users. The entire booking process, from booking to cost calculation and payment, functions completely.
 - **Additional Features:** The application included several advanced features: an interactive map-based dashboard for users (Leaflet.js), a visual real-time spot map for admins, a reviews and ratings system, user profiles and vehicle management, QR code generation for reservations, CSV data export, and email notifications. The UI is fully responsive and has a modern, professional design with custom styling and interactive elements.

Video

- **Link:**
https://drive.google.com/file/d/1d_PMc4RZPbCaOA5ZT5AY36sP3UNn8nv/view?usp=drive_link